Campus Santa Fe

UML Entity-Relationship

Course:

Software Construction and Decision Making

Students:

Jesús José Espinoza Torruco

Vladimir Piñera Reyes

Gonzalo Zamarrón Orrantia

Matriculate:

A01781963

A01786772

A01782739

Delivery Date:

28 de abril

Introduction

This document analyzes the UML entity relationship diagram proposed for Dimension Deck. Each section of the document explains what elements the table stores, why the table is necessary for the game to function, the relation with other tables, and what each field is used for.

Dimension Deck is a real-time roguelite deck builder in which players navigate through rooms across dimensions while creating a card deck via exploration, purchases, fighting different types of enemies, and losing all the progress after dying. That is why this schema must support all of these systems while having the capacity to extend for future changes.

1. User Layer

These tables manage all the elements related to a client. They track the player identity, lifetime statistics, and account data.

1.1 user

Stores the identity record for every registered player. This table functions as the root for the whole diagram.

This table is important because it associates runs, statistics, and progress with a specific person.

Relations

- user → user_stats (1:1): each user has one statistic record.
- user → run (1:N): a user may have multiple runs.

Table

user		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the user.
username	varchar	Displayed name chosen by the player.
email	varchar	Used for login and account recovery.
password	varchar	Hashed credential for authentication.
created_at	timestamp	When the account was registered.
updated_at	timestamp	Last modification to the record.

1.2 user_stats

Tracks the lifetime statistics for a player across its multiple runs. These numbers accumulate over the whole period of the user history.

This table works because it separates the responsibility of the user table, which handles identity, while this table handles the performance data.

Relations

- user_stats → user (1:1): one stats record per user, sharing the same primary key.

Table

user_stats		
Field	**Type**	**Purpose**
user_id (PK, FK)	integer	References user.id.
runs_played	integer	Total number of runs started by this player.
victories	integer	Total number of runs completed successfully.
enemies_slain	integer	Cumulative enemies killed across all runs.
time_played	integer	Total minutes played, used for profile display.
updated_at	timestamp	Last time any stat was modified.

2. Dimension and Run Layer

These tables manage the core run loop of the game: the world the player is in, the state of their current run, and the event log produced during play.

2.1 dimension

Defines the themed worlds available in the game. Each dimension has its own identity, enemy pool, soundtrack, and procedural generation parameters. Currently two dimensions exist: The Old West and The Dark Ages.

This table centralizes the dimension configuration in a single place, making it simple to add new dimensions without changing other systems. The room generation rules (minimum and maximum room count) live here because they are dimension-level properties.

Relations

- dimension → run (1:N): a dimension can host many runs.

- dimension → room (1:N): all rooms in a run inherit their theme from the active dimension.

Table

dimension		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the dimension.
name	varchar	Display name shown on the map screen.
difficulty	float	Multiplier applied to enemy stats and card resistance scaling.
soundtrack	varchar	Path or identifier for the ambient music track.
rooms_min	integer	Minimum number of rooms generated before the boss node.
rooms_max	integer	Maximum number of rooms generated before the boss node.
updated_at	timestamp	Last configuration change to this dimension.

2.2 run

Represents a single attempt by a player. A run begins when the player starts a new game and ends either in victory or death.

The run is the container for a play session. Every room, enemy, player state, and log event that occurs during an attempt is linked back to this record.

Relations

- run → user (N:1): many runs can belong to one user.
- run → dimension (N:1): each run takes place inside one dimension.
- run → run_player (1:1): every run has exactly one player state record.
- run → room (1:N): a run contains many rooms.
- run → run_log (1:N): all events during a run are logged here.

Table

run		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the run.
user_id (FK)	integer	Who is playing.
dimension_id (FK)	integer	Which world.

seed	varchar	Random seed used to generate all rooms and drops.
status	varchar	'active', 'victory', or 'dead'.
started_at	timestamp	When the run began.
updated_at	timestamp	Last state change.

2.3 run_log

Records discrete events that occur during a run, such as a card being played, a room being cleared, or an enemy being killed. Each entry is tied to both the run and the specific room where the event happened.

Relations

- run_log → run (N:1): many log events belong to one run.
- run_log → room (N:1): each event is anchored to the room it occurred in.

Table

run_log		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the log entry.
run_id (FK)	integer	References run.id.
room_id (FK)	integer	References room.id.
event_type	varchar	Structured type: 'card_played', 'enemy_killed', 'room_cleared'.
description	text	Detail for analyzing the run.
created_at	timestamp	When the event occurred.

3. Player State Layer

These tables capture the mutable state of the player during a run: health, credits, deck composition, and slot upgrades purchased mid-run.

3.1 run_player

Holds the live state of the player character during a run. This is distinct from user_stats because it resets on every new run and represents only the current session.

When a player dies, a fresh run_player is created on the next run from default values, cleanly representing the fresh-start mechanic.

Relations

- run_player → run (1:1): one player state per run.
- run_player → deck_card (1:N): the player's current deck is composed of many deck_card rows.
- run_player → slot_upgrade (1:N): all slot capacity purchases during the run.

Table

run_player		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for this player state.
run_id (FK)	integer	References run.id.
hp	float	Current health points.
max_hp	float	Maximum health, modified by upgrades.
credits	integer	Dimensional credits are available to spend in the shop.
active_slots	integer	Current number of active card slots (3 to 5).
auto_slots	integer	Current number of automatic card slots (4 to 8).
resist_penalty	float	Computed card resistance penalty applied to all enemies this run.
updated_at	timestamp	Last state change.

3.2 slot_upgrade

Records each slot capacity purchase made by the player during a run. When a player buys an additional active or automatic card slot at a shrine or shop, a row is inserted here.

Tracking each upgrade individually preserves a full audit trail for the run and helps recalculating the card resistance penalty at any point, since the formula depends on total active and auto slots.

Relations

- slot_upgrade → run_player (N:1): many upgrades can be purchased in one run.
- slot_upgrade → room (N:1): each upgrade is tied to the room where it was purchased.

Table

slot_upgrade		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the upgrade record.

run_player_id (FK)	integer	References run_player.id.
room_id (FK)	integer	References room.id.
type	varchar	'active_slot' or 'auto_slot'.
cost	integer	Credits paid for this upgrade.
new_capacity	integer	Total slot count after this purchase.
updated_at	timestamp	When the upgrade was applied.

4. Room Layer

Rooms are the primary unit of gameplay. Each run is composed of a branching sequence of rooms generated from the dimension configuration. Different room types host different content such as combat, shops, chests, or shrines.

4.1 room

Represents one node in the procedurally generated room graph for a run. Room types include combat, chest, shop, and boss.

Every interactive element in the game (enemies, shop items, log events, and upgrades) hangs off a room foreign key. This makes room for the central hub of the gameplay layer.

Relations

- room → run (N:1): many rooms belong to one run.
- room → dimension (N:1): the room inherits its visual theme from the dimension.
- room → enemy (1:N): combat rooms spawn multiple enemies.
- room → shop_item (1:N): shop rooms contain multiple purchasable items.
- room → slot_upgrade (1:N): upgrades purchased in this room reference it.
- room → run_log (1:N): all events in this room are logged against it.

Table

room		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the room.
run_id (FK)	integer	References run.id.
dimension_id (FK)	integer	References dimension.id for the visual theme.
number	integer	Position in the run sequence.
type	varchar	'combat', 'chest', 'shop', 'shrine', 'glitch', or 'boss'.

cleared	boolean	True once the room's exit condition is met.
width_px	integer	Room width in pixels (default 480).
height_px	integer	Room height in pixels (default 352).
updated_at	timestamp	Last state change.

4.2 shop_item

Stores the inventory of a shop room. Supports any item category the shop might sell.

Relations

- shop_item → room (N:1): many items belong to one shop room.

Table

shop_item		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier for the shop listing.
room_id (FK)	integer	References room.id.
item_type	varchar	'card', 'slot_upgrade', or 'removal'.
item_ref_id	varchar	The id of the card being sold.
price	integer	Cost in dimensional credits.
sold	boolean	True once the player has purchased this item.
updated_at	timestamp	Last state change.

5. Card Layer

Cards are the primary gameplay mechanic. The card layer stores the static catalog of all cards, their effects, their subtype data, and the player's current deck during a run.

5.1 card

This is a table that helps represent card definitions. When a player adds a card to their deck, a deck_card row is created pointing to this table.

Separating the card definition from the player's deck instance means card data is written once and read many times. Rarity, cost, and cooldown all live here and are inherited by every deck instance of that card.

Relations

- card → deck_card (1:N): many deck instances can reference the same card definition.
- card → card_effect (1:N): a card can have multiple effects.
- card → damage_card (1:0..1): a card optionally extends as a damage type.
- card → heal_card (1:0..1): a card optionally extends as a heal type.
- card → defense_card (1:0..1): a card optionally extends as a defense type.

Table

card		
Field	**Type**	**Purpose**
id (PK)	varchar	Unique string identifier
name	varchar	Display name shown on the card.
rarity	varchar	'common', 'rare', 'epic', or 'legendary'.
category	varchar	'active' or 'automatic'. Determines which slot the card occupies.
type	varchar	'damage', 'heal', 'defense'
cost	integer	Credits required to purchase from a shop.
cooldown	float	Seconds before the card can be used again after activation.
updated_at	timestamp	Last change to the card's base definition.

5.2 card_effect

Stores the discrete effects attached to a card, such as a damage value, a status duration, or a chance to apply poison. A single card can have multiple effects.

Relations

- card_effect → card (N:1): many effects belong to one card.

Table

card_effect		
Field	**Type**	**Purpose**
id (PK)	integer	Unique identifier.
card_id (FK)	varchar	References card.id.
effect_type	varchar	'damage', 'slow', 'poison', 'shield'.
value	float	Magnitude of the effect
duration	float	How long the effect lasts in seconds.
apply_chance	float	Probability 0.0 to 1.0 that the effect triggers on use.
updated_at	timestamp	Last modification.

5.3 damage_card / heal_card / defense_card

Three specialization tables that extend the base card record with type-specific fields using the same primary key as their parent. A damage card stores damage amount and AOE flag; a heal card stores heal amount and over-time flag; a defense card stores defense amount, duration, and knockback resistance.

Relations

- damage_card → card (1:1): shares card.id as primary key.
- heal_card → card (1:1): same pattern.
- defense_card → card (1:1): same pattern.

Table

damage_card / heal_card / defense_card		
Field	**Type**	**Purpose**
card_id (PK, FK)	varchar	Shared with card.id.
dmg_amount / heal_amount / def_amount	float	Core numeric value for the card's primary effect.
dmg_type	varchar	'physical', 'fire', 'poison'.
is_aoe	boolean	(damage only) Whether the effect hits an area.
over_time	boolean	(heal only) Whether healing is applied gradually over time.
duration	float	(defense only) How long the defensive effect lasts in seconds.
knockback_res	float	(defense only) Knockback resistance value granted to the player.
updated_at	timestamp	Last modification.

5.4 deck_card

Represents a card instance inside a specific player's deck during a run.

Relations

- deck_card → run_player (N:1): many cards belong to one player's run state.
- deck_card → card (N:1): many deck instances can reference the same card definition.

Table

deck_card		
Field	**Type**	**Purpose**

id (PK)	integer	Unique identifier for this deck slot.
run_player_id (FK)	integer	References run_player.id.
card_id (FK)	varchar	Which card this slot holds.
slot_type	varchar	'active' or 'automatic'.
level	integer	Upgrade level: 1=base, 2=upgraded, 3=max.
is_starter	boolean	True if this card was part of the initial starting deck.
updated_at	timestamp	Last change.

6. Enemy Layer

The enemy layer uses a base table plus three specialization tables to model the three enemy types: swarm, tank, and ranged.

6.1 enemy

The base record for every enemy instance in the game. Stores properties shared by all archetypes: which room the enemy is in, its name, archetype label, sprite size, current HP, and status.

Relations

- enemy → room (N:1): many enemies belong to one room.
- enemy → swarm_enemy (1:0..1): an enemy optionally extends as swarm type.
- enemy → tank_enemy (1:0..1): an enemy optionally extends as tank type.
- enemy → ranged_enemy (1:0..1): an enemy optionally extends as ranged type.

Table

enemy		
Field	**Type**	**Purpose**
id (PK)	varchar	Unique identifier, typically a generated string per instance.
room_id (FK)	integer	The room this enemy spawned in.
name	varchar	Display name.
archetype	varchar	'swarm', 'tank', or 'ranged'.
sprite_px	integer	Sprite size in pixels.
hp	float	Current health points.
alive	boolean	False once the enemy has been killed.
updated_at	timestamp	Last state change.

6.2 swarm_enemy

Extends the base enemy record with swarm-specific combat properties. Swarm enemies are small, fast units that pressure the player by crowding the playable space and dealing contact damage.

Relations

- swarm_enemy → enemy (1:1): shares enemy.id as primary key.

Table

swarm_enemy		
Field	**Type**	**Purpose**
enemy_id (PK, FK)	varchar	Shared with enemy.id.
speed	float	Movement speed when pursuing the player.
contact_dmg	float	Damage dealt to the player on direct contact.
crowd_priority	integer	Determines crowding behavior when multiple swarm enemies converge on the player.
updated_at	timestamp	Last modification.

6.3 tank_enemy

Tank enemies are large, slow units that absorb significant damage and punish the player for staying at close range.

Relations

- tank_enemy → enemy (1:1): shares enemy.id as primary key.

Table

tank_enemy		
Field	**Type**	**Purpose**
enemy_id (PK, FK)	varchar	Shared with enemy.id.
knockback_res	float	Percentage resistance to player knockback effects.
melee_dmg	float	Damage dealt on a melee swing.

armor	float	Flat damage reduction applied to all incoming hits.
updated_at	timestamp	Last modification.

6.4 ranged_enemy

Ranged enemies maintain distance from the player and fire projectiles on a cooldown, forcing constant movement. Projectile speed, fire cooldown, and preferred engagement distance are properties that only make sense for ranged enemies.

Relations

- ranged_enemy → enemy (1:1): shares enemy.id as primary key.

Table

ranged_enemy		
Field	**Type**	**Purpose**
enemy_id (PK, FK)	varchar	Shared with enemy.id.
proj_speed	float	Speed of the projectile fired by this enemy.
fire_cooldown	float	Seconds between consecutive projectile shots.
min_distance	float	Preferred minimum distance maintained from the player.
accuracy	float	Tracking the precision of the projectile aimed at the player.
updated_at	timestamp	Last modification.